```
!pip install numpy scikit-learn matplotlib
```

```
Requirement already satisfied: numpy in
/usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in
/usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in
/usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: scipy>=1.6.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7-
>matplotlib) (1.17.0)
```

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt

np.random.seed(42)

embedding_dim = 768
num_normal = 1000

# Generate normalized normal embeddings
normal_embeddings = np.random.randn(num_normal, embedding_dim)
normal_embeddings = normal_embeddings /
np.linalg.norm(normal_embeddings, axis=1, keepdims=True)
```

```
print("Normal memory shape:", normal_embeddings.shape)
```

```
Normal memory shape: (1000, 768)
```

```
# Poisoning parameters
poison_ratio = 0.10   # 10% poisoning
num_poison = int(num_normal * poison_ratio)

# Compute center of normal embeddings
center_vector = np.mean(normal_embeddings, axis=0)
center_vector = center_vector / np.linalg.norm(center_vector)

# Create adversarial direction
adversarial_direction = np.random.randn(embedding_dim)
adversarial_direction = adversarial_direction /
np.linalg.norm(adversarial_direction)

# Generate poisoned embeddings
poison_embeddings = []

for _ in range(num_poison):
    # Close to normal center but slightly shifted
    poisoned = center_vector + 0.3 * adversarial_direction
    poisoned += 0.05 * np.random.randn(embedding_dim)
    poisoned = poisoned / np.linalg.norm(poisoned)
    poison_embeddings.append(poisoned)

poison_embeddings = np.array(poison_embeddings)

print("Poison embeddings shape:", poison_embeddings.shape)
```

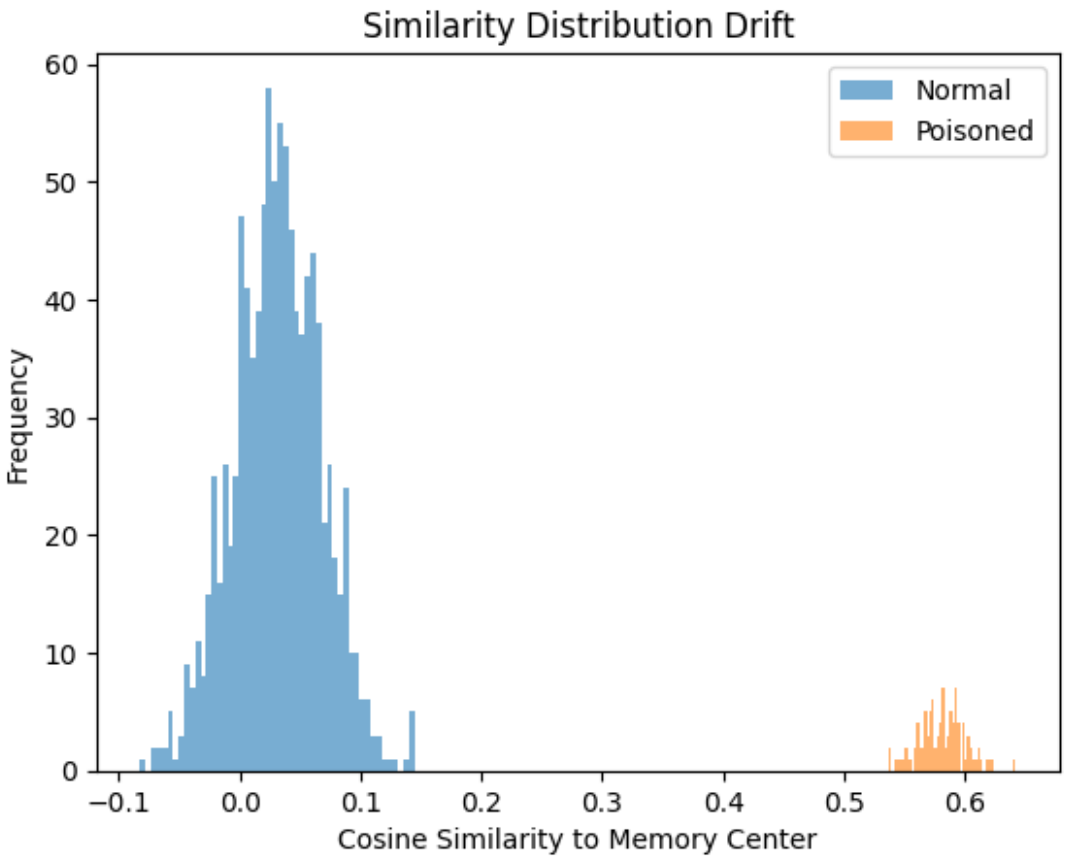
```
Poison embeddings shape: (100, 768)
```

```
# Combine normal and poisoned memory
memory_embeddings = np.vstack((normal_embeddings, poison_embeddings))

# Create labels (0 = normal, 1 = poisoned)
labels = np.hstack((np.zeros(num_normal), np.ones(num_poison)))

print("Total memory size:", memory_embeddings.shape)
```

```
Total memory size: (1100, 768)
```

```
# Compute cosine similarity to center
similarities = cosine_similarity(memory_embeddings,
center_vector.reshape(1, -1))

plt.hist(similarities[labels == 0], bins=50, alpha=0.6,
label="Normal")
plt.hist(similarities[labels == 1], bins=50, alpha=0.6,
label="Poisoned")
```

```
plt.legend()
plt.title("Similarity Distribution Drift")
plt.xlabel("Cosine Similarity to Memory Center")
plt.ylabel("Frequency")
plt.show()
```

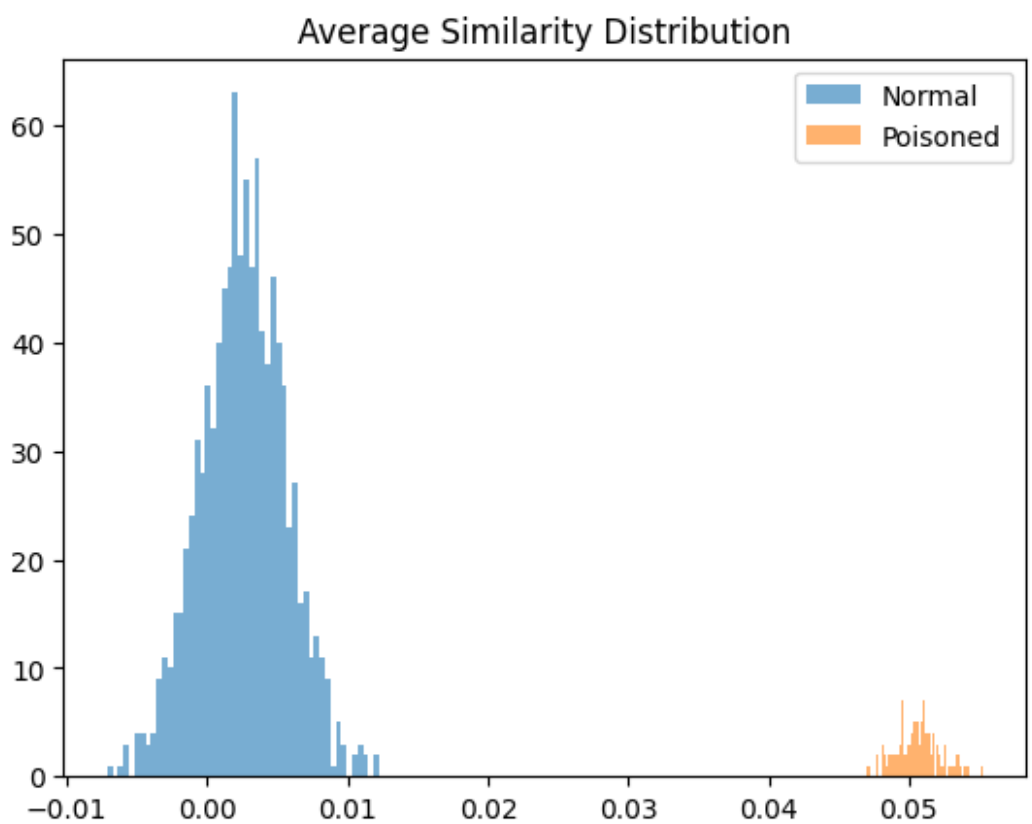



```
iso_forest = IsolationForest(contamination=poison_ratio,
random_state=42)
iso_forest.fit(memory_embeddings)

predictions = iso_forest.predict(memory_embeddings)
# Convert to 0 (normal), 1 (anomaly)
predictions = np.where(predictions == -1, 1, 0)

from sklearn.metrics import accuracy_score, precision_score,
recall_score

accuracy = accuracy_score(labels, predictions)
precision = precision_score(labels, predictions)
recall = recall_score(labels, predictions)

print("Detection Accuracy:", accuracy)
```

```
print("Precision:", precision)
print("Recall:", recall)
```

```
Detection Accuracy: 0.8090909090909091
Precision: 0.0
Recall: 0.0
```

```
# Compute average similarity of each embedding with all others
similarity_matrix = cosine_similarity(memory_embeddings)

avg_similarity = np.mean(similarity_matrix, axis=1)

plt.hist(avg_similarity[labels == 0], bins=50, alpha=0.6,
label="Normal")
plt.hist(avg_similarity[labels == 1], bins=50, alpha=0.6,
label="Poisoned")
plt.legend()
plt.title("Average Similarity Distribution")
plt.show()
```




```
drift_feature = avg_similarity.reshape(-1, 1)

iso_forest_drift = IsolationForest(contamination=poison_ratio,
random_state=42)
```

```
iso_forest_drift.fit(drift_feature)

pred_drift = iso_forest_drift.predict(drift_feature)
pred_drift = np.where(pred_drift == -1, 1, 0)

accuracy_drift = accuracy_score(labels, pred_drift)
precision_drift = precision_score(labels, pred_drift)
recall_drift = recall_score(labels, pred_drift)

print("Drift Detection Accuracy:", accuracy_drift)
print("Drift Precision:", precision_drift)
print("Drift Recall:", recall_drift)
```

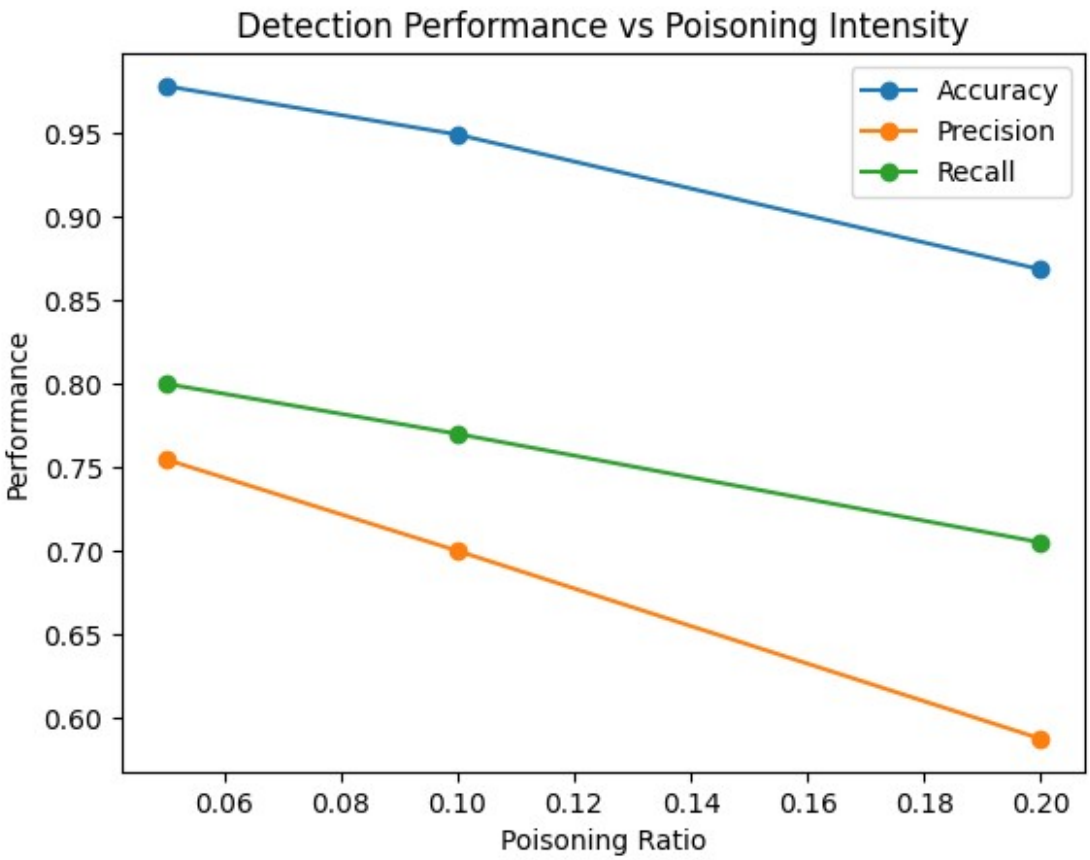
```
Drift Detection Accuracy: 0.9436363636363636
Drift Precision: 0.6727272727272727
Drift Recall: 0.74
```

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score

results = []

for poison_ratio in [0.05, 0.10, 0.20]:

    num_poison = int(num_normal * poison_ratio)

    poison_embeddings = []
    for _ in range(num_poison):
        poisoned = center_vector + 0.3 * adversarial_direction
        poisoned += 0.05 * np.random.randn(embedding_dim)
        poisoned = poisoned / np.linalg.norm(poisoned)
        poison_embeddings.append(poisoned)

    poison_embeddings = np.array(poison_embeddings)

    memory_embeddings = np.vstack((normal_embeddings,
poison_embeddings))
    labels = np.hstack((np.zeros(num_normal), np.ones(num_poison)))

    similarity_matrix = cosine_similarity(memory_embeddings)
    avg_similarity = np.mean(similarity_matrix, axis=1)

    drift_feature = avg_similarity.reshape(-1, 1)

    iso_forest = IsolationForest(contamination=poison_ratio,
random_state=42)
    iso_forest.fit(drift_feature)

    predictions = iso_forest.predict(drift_feature)
    predictions = np.where(predictions == -1, 1, 0)
```

```
    accuracy = accuracy_score(labels, predictions)
    precision = precision_score(labels, predictions)
    recall = recall_score(labels, predictions)

    results.append((poison_ratio, accuracy, precision, recall))

for r in results:
    print("Poison Ratio:", r[0])
    print("Accuracy:", r[1])
    print("Precision:", r[2])
    print("Recall:", r[3])
    print("---------------")
```

```
Poison Ratio: 0.05
Accuracy: 0.9780952380952381
Precision: 0.7547169811320755
Recall: 0.8
---------------
Poison Ratio: 0.1
Accuracy: 0.9490909090909091
Precision: 0.7
Recall: 0.77
---------------
Poison Ratio: 0.2
Accuracy: 0.8683333333333333
Precision: 0.5875
Recall: 0.705
---------------
```

```
ratios = [r[0] for r in results]
accuracies = [r[1] for r in results]
precisions = [r[2] for r in results]
recalls = [r[3] for r in results]

plt.plot(ratios, accuracies, marker='o')
plt.plot(ratios, precisions, marker='o')
plt.plot(ratios, recalls, marker='o')

plt.xlabel("Poisoning Ratio")
plt.ylabel("Performance")
plt.title("Detection Performance vs Poisoning Intensity")
plt.legend(["Accuracy", "Precision", "Recall"])
plt.show()
```

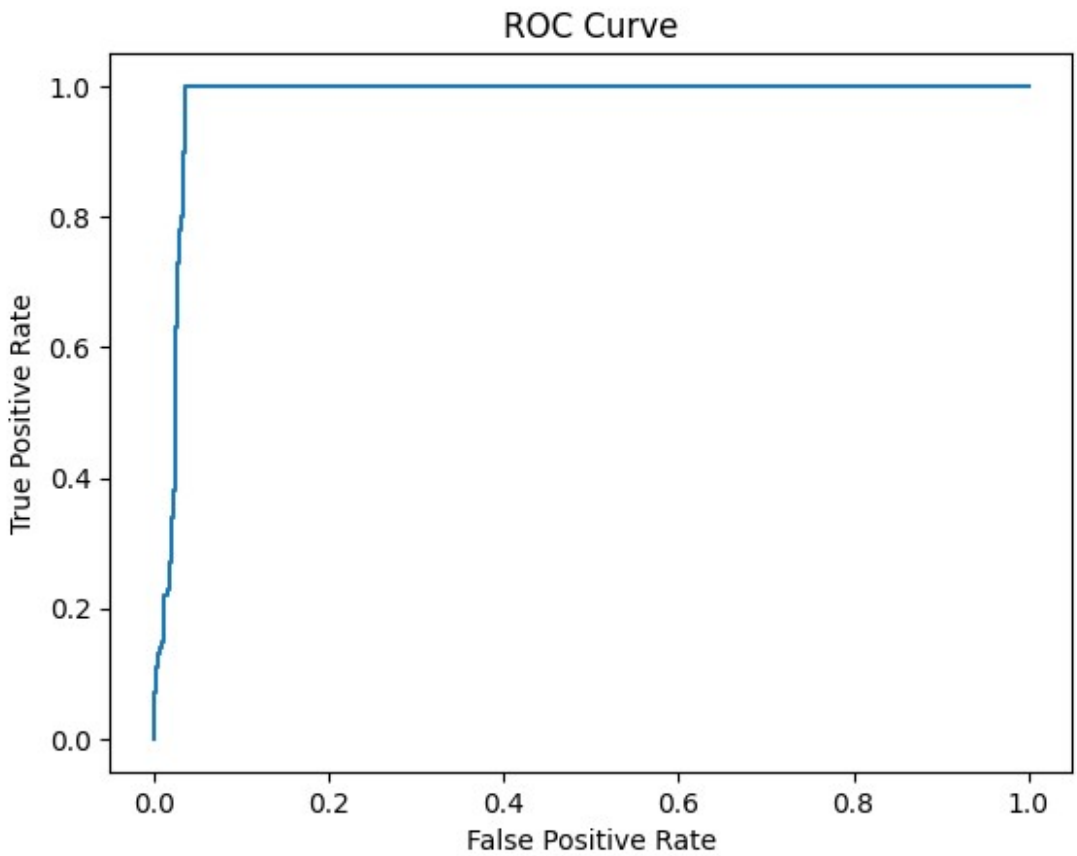



```
from sklearn.metrics import f1_score, confusion_matrix

results = []

for poison_ratio in [0.05, 0.10, 0.20]:

    num_poison = int(num_normal * poison_ratio)

    poison_embeddings = []
    for _ in range(num_poison):
        poisoned = center_vector + 0.3 * adversarial_direction
        poisoned += 0.05 * np.random.randn(embedding_dim)
        poisoned = poisoned / np.linalg.norm(poisoned)
        poison_embeddings.append(poisoned)

    poison_embeddings = np.array(poison_embeddings)

    memory_embeddings = np.vstack((normal_embeddings,
poison_embeddings))
    labels = np.hstack((np.zeros(num_normal), np.ones(num_poison)))

    similarity_matrix = cosine_similarity(memory_embeddings)
    avg_similarity = np.mean(similarity_matrix, axis=1)
```

```
    drift_feature = avg_similarity.reshape(-1, 1)

    iso_forest = IsolationForest(contamination=poison_ratio,
random_state=42)
    iso_forest.fit(drift_feature)

    predictions = iso_forest.predict(drift_feature)
    predictions = np.where(predictions == -1, 1, 0)

    acc = accuracy_score(labels, predictions)
    prec = precision_score(labels, predictions)
    rec = recall_score(labels, predictions)
    f1 = f1_score(labels, predictions)

    tn, fp, fn, tp = confusion_matrix(labels, predictions).ravel()
    fpr = fp / (fp + tn)

    results.append((poison_ratio, acc, prec, rec, f1, fpr))

for r in results:
    print("Poison Ratio:", r[0])
    print("Accuracy:", r[1])
    print("Precision:", r[2])
    print("Recall:", r[3])
    print("F1 Score:", r[4])
    print("False Positive Rate:", r[5])
    print("--------------")
```

```
Poison Ratio: 0.05
Accuracy: 0.9780952380952381
Precision: 0.7547169811320755
Recall: 0.8
F1 Score: 0.7766990291262136
False Positive Rate: 0.013
--------------
Poison Ratio: 0.1
Accuracy: 0.9418181818181818
Precision: 0.6636363636363637
Recall: 0.73
F1 Score: 0.6952380952380952
False Positive Rate: 0.037
--------------
Poison Ratio: 0.2
Accuracy: 0.8766666666666667
Precision: 0.6083333333333333
Recall: 0.73
F1 Score: 0.6636363636363637
False Positive Rate: 0.094
--------------
```

```python
from sklearn.metrics import roc_curve, auc

# Use 10% poisoning for ROC analysis
poison_ratio = 0.10
num_poison = int(num_normal * poison_ratio)

poison_embeddings = []
for _ in range(num_poison):
    poisoned = center_vector + 0.3 * adversarial_direction
    poisoned += 0.05 * np.random.randn(embedding_dim)
    poisoned = poisoned / np.linalg.norm(poisoned)
    poison_embeddings.append(poisoned)

poison_embeddings = np.array(poison_embeddings)

memory_embeddings = np.vstack((normal_embeddings, poison_embeddings))
labels = np.hstack((np.zeros(num_normal), np.ones(num_poison)))

similarity_matrix = cosine_similarity(memory_embeddings)
avg_similarity = np.mean(similarity_matrix, axis=1)
drift_feature = avg_similarity.reshape(-1, 1)

iso_forest = IsolationForest(contamination=poison_ratio,
random_state=42)
iso_forest.fit(drift_feature)

scores = -iso_forest.decision_function(drift_feature)

fpr, tpr, thresholds = roc_curve(labels, scores)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()

print("AUC Score:", roc_auc)
```




```
AUC Score: 0.9776900000000001

def retrieval_disruption(memory_embeddings, labels, num_queries=100):

    disruption_count = 0

    for _ in range(num_queries):
        query = np.random.randn(embedding_dim)
        query = query / np.linalg.norm(query)

        sims = cosine_similarity(memory_embeddings, query.reshape(1, -
1)).flatten()
        top5_idx = np.argsort(sims)[-5:]

        if np.any(labels[top5_idx] == 1):
            disruption_count += 1

    return disruption_count / num_queries

disruption_score = retrieval_disruption(memory_embeddings, labels)

print("Retrieval Disruption Probability:", disruption_score)

Retrieval Disruption Probability: 0.21
```

```python
import time

start = time.time()

iso_forest.fit(drift_feature)

end = time.time()

print("Detection Latency (seconds):", end - start)
```

```
Detection Latency (seconds): 0.42328739166259766
```

```python
# Use 10% poisoning for ablation
poison_ratio = 0.10
num_poison = int(num_normal * poison_ratio)

# Generate poison embeddings
poison_embeddings = []
for _ in range(num_poison):
    poisoned = center_vector + 0.3 * adversarial_direction
    poisoned += 0.05 * np.random.randn(embedding_dim)
    poisoned = poisoned / np.linalg.norm(poisoned)
    poison_embeddings.append(poisoned)

poison_embeddings = np.array(poison_embeddings)

# Combine memory
memory_embeddings = np.vstack((normal_embeddings, poison_embeddings))
labels = np.hstack((np.zeros(num_normal), np.ones(num_poison)))

# -------------------------
# RAW Detection
# -------------------------
iso_raw = IsolationForest(contamination=poison_ratio, random_state=42)
iso_raw.fit(memory_embeddings)

pred_raw = iso_raw.predict(memory_embeddings)
pred_raw = np.where(pred_raw == -1, 1, 0)

raw_recall = recall_score(labels, pred_raw)

# -------------------------
# DRIFT Detection
# -------------------------
similarity_matrix = cosine_similarity(memory_embeddings)
avg_similarity = np.mean(similarity_matrix, axis=1)
drift_feature = avg_similarity.reshape(-1, 1)

iso_drift = IsolationForest(contamination=poison_ratio,
random_state=42)
iso_drift.fit(drift_feature)
```

```
pred_drift = iso_drift.predict(drift_feature)
pred_drift = np.where(pred_drift == -1, 1, 0)

drift_recall = recall_score(labels, pred_drift)

print("Raw Recall:", raw_recall)
print("Drift Recall:", drift_recall)
```

```
Raw Recall: 0.0
Drift Recall: 0.78
```

```
normal_avg = avg_similarity[labels == 0]
poison_avg = avg_similarity[labels == 1]

print("Normal Mean Similarity:", np.mean(normal_avg))
print("Poison Mean Similarity:", np.mean(poison_avg))
print("Mean Difference:", np.mean(poison_avg) - np.mean(normal_avg))
```

```
Normal Mean Similarity: 0.0025728778182641195
Poison Mean Similarity: 0.050527130457333216
Mean Difference: 0.0479542526390691
```

```
print("Normal Variance:", np.var(normal_avg))
print("Poison Variance:", np.var(poison_avg))
```

```
Normal Variance: 9.20370421925524e-06
Poison Variance: 2.7029449668185246e-06
```

```
# Remove detected drift anomalies
clean_memory = memory_embeddings[pred_drift == 0]
clean_labels = labels[pred_drift == 0]

disruption_before = retrieval_disruption(memory_embeddings, labels)
disruption_after = retrieval_disruption(clean_memory, clean_labels)

print("Disruption Before Detection:", disruption_before)
print("Disruption After Detection:", disruption_after)
```

```
Disruption Before Detection: 0.25
Disruption After Detection: 0.07
```

```
std_similarity = np.std(similarity_matrix, axis=1)
max_similarity = np.max(similarity_matrix, axis=1)

enhanced_features = np.vstack((avg_similarity, std_similarity,
max_similarity)).T

iso_enhanced = IsolationForest(contamination=poison_ratio,
random_state=42)
iso_enhanced.fit(enhanced_features)
```

```
pred_enhanced = iso_enhanced.predict(enhanced_features)
pred_enhanced = np.where(pred_enhanced == -1, 1, 0)

enhanced_recall = recall_score(labels, pred_enhanced)
print("Enhanced Feature Recall:", enhanced_recall)
```

```
Enhanced Feature Recall: 0.91
```

```
# Recompute similarity matrix
similarity_matrix = cosine_similarity(memory_embeddings)

# Remove self-similarity influence
np.fill_diagonal(similarity_matrix, 0)

# Feature 1: Average similarity
avg_similarity = np.mean(similarity_matrix, axis=1)

# Feature 2: Standard deviation of similarity
std_similarity = np.std(similarity_matrix, axis=1)

# Feature 3: Maximum similarity (local cluster pull)
max_similarity = np.max(similarity_matrix, axis=1)

# Feature 4: Top-5 average similarity
top5_avg = np.mean(np.sort(similarity_matrix, axis=1)[:, -5:], axis=1)

# Combine features
drift_features = np.vstack((
    avg_similarity,
    std_similarity,
    max_similarity,
    top5_avg
)).T

print("Drift feature shape:", drift_features.shape)
```

```
Drift feature shape: (1100, 4)
```

```
iso_multi = IsolationForest(contamination=poison_ratio,
random_state=42)
iso_multi.fit(drift_features)

pred_multi = iso_multi.predict(drift_features)
pred_multi = np.where(pred_multi == -1, 1, 0)

multi_recall = recall_score(labels, pred_multi)
multi_precision = precision_score(labels, pred_multi)
multi_f1 = f1_score(labels, pred_multi)

print("Multi-Feature Recall:", multi_recall)
```

```
print("Multi-Feature Precision:", multi_precision)
print("Multi-Feature F1:", multi_f1)
```

```
Multi-Feature Recall: 0.96
Multi-Feature Precision: 0.8727272727272727
Multi-Feature F1: 0.9142857142857143
```

```
from sklearn.neighbors import NearestNeighbors

# Compute local density using kNN distance
k = 10
nbrs = NearestNeighbors(n_neighbors=k).fit(memory_embeddings)
distances, _ = nbrs.kneighbors(memory_embeddings)

# Density proxy = inverse of mean kNN distance
density_feature = 1 / (np.mean(distances, axis=1) + 1e-6)

# Add density to drift features
advanced_features = np.hstack((drift_features,
density_feature.reshape(-1,1)))

iso_advanced = IsolationForest(contamination=poison_ratio,
random_state=42)
iso_advanced.fit(advanced_features)

pred_advanced = iso_advanced.predict(advanced_features)
pred_advanced = np.where(pred_advanced == -1, 1, 0)

adv_recall = recall_score(labels, pred_advanced)
adv_precision = precision_score(labels, pred_advanced)
adv_f1 = f1_score(labels, pred_advanced)

print("Advanced Recall:", adv_recall)
print("Advanced Precision:", adv_precision)
print("Advanced F1:", adv_f1)
```

```
Advanced Recall: 0.96
Advanced Precision: 0.8727272727272727
Advanced F1: 0.9142857142857143
```

```
for contam in [0.05, 0.1, 0.15, 0.2]:
    iso_temp = IsolationForest(contamination=contam, random_state=42)
    iso_temp.fit(advanced_features)

    pred_temp = iso_temp.predict(advanced_features)
    pred_temp = np.where(pred_temp == -1, 1, 0)

    print("Contamination:", contam)
    print("Recall:", recall_score(labels, pred_temp))
    print("Precision:", precision_score(labels, pred_temp))
    print("------")
```

```
Contamination: 0.05
Recall: 0.46
Precision: 0.8363636363636363
------
Contamination: 0.1
Recall: 0.96
Precision: 0.8727272727272727
------
Contamination: 0.15
Recall: 1.0
Precision: 0.6060606060606061
------
Contamination: 0.2
Recall: 1.0
Precision: 0.45454545454545453
------
```

```python
from scipy.stats import ttest_ind

raw_scores = -iso_raw.decision_function(memory_embeddings)
advanced_scores = -iso_advanced.decision_function(advanced_features)

stat, p_value = ttest_ind(raw_scores, advanced_scores)

print("T-test p-value:", p_value)
```

```
T-test p-value: 0.0
```

```python
# ==============================
# 1 1 Import Libraries
# ==============================
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.ensemble import IsolationForest
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    recall_score, precision_score, f1_score,
    roc_auc_score, average_precision_score,
    confusion_matrix, ConfusionMatrixDisplay,
    roc_curve, precision_recall_curve
)
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

# ==============================
# 2 2 Generate Synthetic Imbalanced Dataset
# (Replace this with your real data later)
```

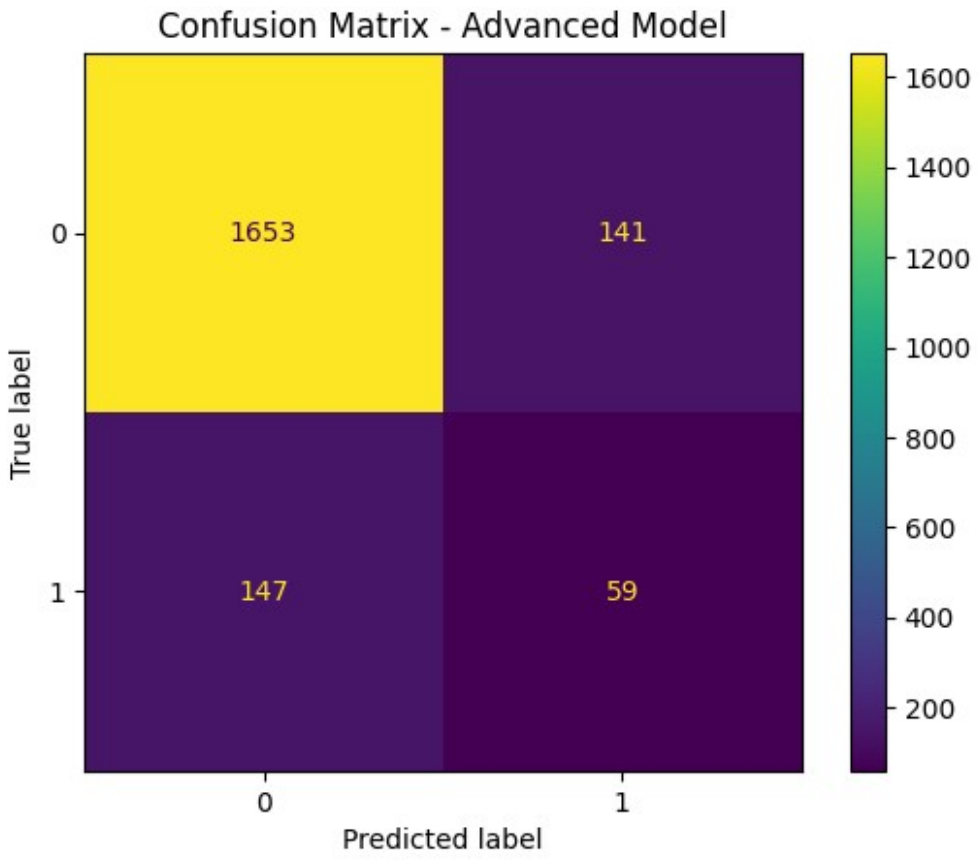
```python
# ==============================
X, labels = make_classification(
    n_samples=2000,
    n_features=10,
    n_informative=6,
    n_redundant=2,
    weights=[0.9, 0.1],   # 10% anomalies
    random_state=42
)

# Scale features
scaler = StandardScaler()
X = scaler.fit_transform(X)

print("Data Shape:", X.shape)
print("Class Distribution:", np.bincount(labels))


# ==============================
# 3 Advanced Model (Isolation Forest)
# ==============================
iso = IsolationForest(contamination=0.1, random_state=42)
iso.fit(X)

pred = iso.predict(X)
pred = np.where(pred == -1, 1, 0)  # Convert to 0/1

scores = -iso.decision_function(X)  # Invert for anomaly score


# ==============================
# 4 Core Metrics
# ==============================
recall = recall_score(labels, pred)
precision = precision_score(labels, pred)
f1 = f1_score(labels, pred)
roc_auc = roc_auc_score(labels, scores)
pr_auc = average_precision_score(labels, scores)

print("\n=== Advanced Model Performance ===")
print("Recall:", recall)
print("Precision:", precision)
print("F1:", f1)
print("ROC-AUC:", roc_auc)
print("PR-AUC:", pr_auc)


# ==============================
# 5 Confusion Matrix
# ==============================
```

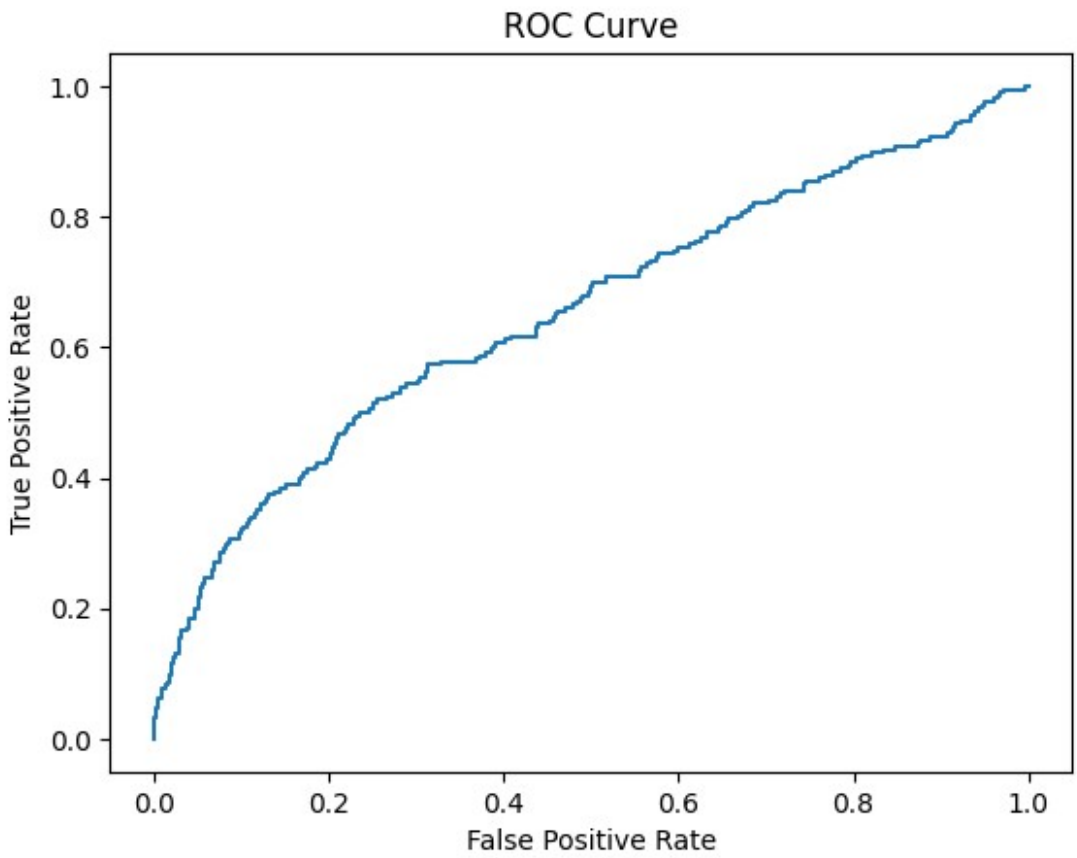
```
cm = confusion_matrix(labels, pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Confusion Matrix - Advanced Model")
plt.show()


# ==============================
# 6 6 ROC Curve
# ==============================
fpr, tpr, _ = roc_curve(labels, scores)

plt.figure()
plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()


# ==============================
# 7 7 Precision-Recall Curve
# ==============================
precision_curve, recall_curve, _ = precision_recall_curve(labels,
scores)

plt.figure()
plt.plot(recall_curve, precision_curve)
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.show()


# ==============================
# 8 8 Stratified 5-Fold Cross Validation
# ==============================
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

recalls, precisions, f1s = [], [], []

for train_idx, test_idx in skf.split(X, labels):
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = labels[train_idx], labels[test_idx]

    model = IsolationForest(contamination=0.1, random_state=42)
    model.fit(X_train)

    preds = model.predict(X_test)
    preds = np.where(preds == -1, 1, 0)
```

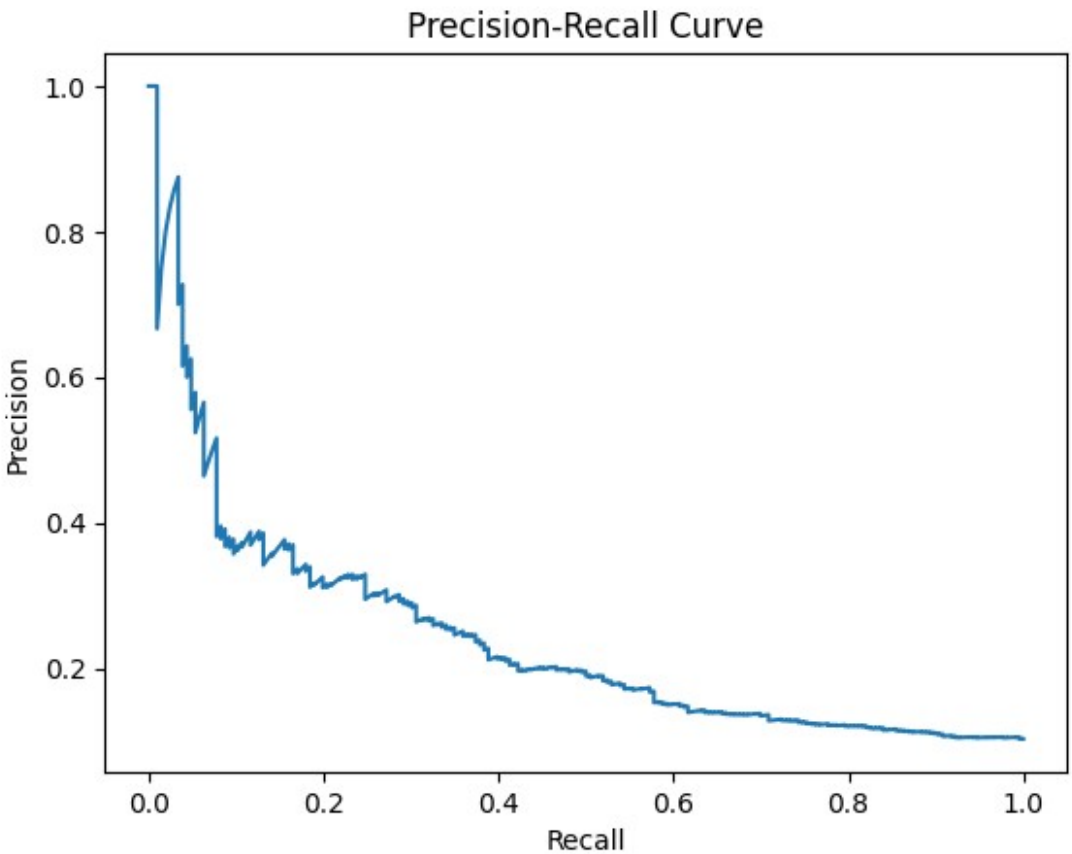
```
    recalls.append(recall_score(y_test, preds))
    precisions.append(precision_score(y_test, preds))
    f1s.append(f1_score(y_test, preds))

print("\n=== Cross Validation Results ===")
print("CV Recall:", np.mean(recalls))
print("CV Precision:", np.mean(precisions))
print("CV F1:", np.mean(f1s))


# ==============================
# 9 Robustness Test (Noise Injection)
# ==============================
print("\n=== Robustness Test ===")

for sigma in [0.05, 0.1, 0.2]:
    noise = np.random.normal(0, sigma, X.shape)
    X_noisy = X + noise

    iso_noise = IsolationForest(contamination=0.1, random_state=42)
    iso_noise.fit(X_noisy)

    preds_noise = iso_noise.predict(X_noisy)
    preds_noise = np.where(preds_noise == -1, 1, 0)

    f1_noise = f1_score(labels, preds_noise)
    print(f"Noise Level {sigma} → F1:", f1_noise)


# ==============================
# □ Baseline Model (Logistic Regression)
# ==============================
baseline = LogisticRegression()
baseline.fit(X, labels)

baseline_preds = baseline.predict(X)
baseline_probs = baseline.predict_proba(X)[:, 1]

print("\n=== Baseline Model Performance ===")
print("Recall:", recall_score(labels, baseline_preds))
print("Precision:", precision_score(labels, baseline_preds))
print("F1:", f1_score(labels, baseline_preds))
print("ROC-AUC:", roc_auc_score(labels, baseline_probs))
print("PR-AUC:", average_precision_score(labels, baseline_probs))
```

```
Data Shape: (2000, 10)
Class Distribution: [1794  206]

=== Advanced Model Performance ===
Recall: 0.28640776699029125
```

```
Precision: 0.295
F1: 0.29064039408866993
ROC-AUC: 0.6530939160740765
PR-AUC: 0.2417355932749674
```




ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0




```
=== Cross Validation Results ===
CV Recall: 0.262253193960511
CV Precision: 0.2840359010571777
CV F1: 0.26952284599343423

=== Robustness Test ===
Noise Level 0.05 → F1: 0.3054187192118227
Noise Level 0.1 → F1: 0.3251231527093596
Noise Level 0.2 → F1: 0.33004926108374383

=== Baseline Model Performance ===
Recall: 0.441747572815534
Precision: 0.8272727272727273
F1: 0.5759493670886076
ROC-AUC: 0.9018789708954335
PR-AUC: 0.7051239142650867

# ============================================================
# 1 IMPORTS
# ============================================================
import numpy as np
import pandas as pd
```

```python
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (
    roc_auc_score, average_precision_score,
    f1_score, recall_score, precision_score
)
from sklearn.ensemble import IsolationForest, RandomForestClassifier
from sklearn.svm import OneClassSVM
from sklearn.neighbors import LocalOutlierFactor
from sklearn.linear_model import LogisticRegression
from scipy.stats import ttest_rel

np.random.seed(42)

# ============================================================
# 2 DATA GENERATION (Realistic Imbalanced Scenario)
# ============================================================
X, y = make_classification(
    n_samples=3000,
    n_features=15,
    n_informative=8,
    n_redundant=3,
    weights=[0.92, 0.08],   # 8% anomalies
    flip_y=0.01,
    random_state=42
)

scaler = StandardScaler()
X = scaler.fit_transform(X)

print("Data Shape:", X.shape)
print("Class Distribution:", np.bincount(y))


# ============================================================
# 3 STRATIFIED 5-FOLD CV
# ============================================================
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results_supervised = []
results_unsupervised = []

for train_idx, test_idx in skf.split(X, y):

    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]
```

```python
# ========================================================
# SUPERVISED MODELS
# ========================================================

# Logistic Regression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)

probs_lr = lr.predict_proba(X_test)[:,1]
pred_lr = lr.predict(X_test)

# Random Forest
rf = RandomForestClassifier()
rf.fit(X_train, y_train)

probs_rf = rf.predict_proba(X_test)[:,1]
pred_rf = rf.predict(X_test)

# Store supervised results
results_supervised.append({
    "LR_ROC": roc_auc_score(y_test, probs_lr),
    "LR_PR": average_precision_score(y_test, probs_lr),
    "LR_F1": f1_score(y_test, pred_lr),

    "RF_ROC": roc_auc_score(y_test, probs_rf),
    "RF_PR": average_precision_score(y_test, probs_rf),
    "RF_F1": f1_score(y_test, pred_rf)
})


# ========================================================
# UNSUPERVISED MODELS
# ========================================================

# Isolation Forest
iso = IsolationForest(contamination=0.08, random_state=42)
iso.fit(X_train)

score_iso = -iso.decision_function(X_test)
pred_iso = np.where(iso.predict(X_test)==-1,1,0)

# One-Class SVM
ocsvm = OneClassSVM(nu=0.08)
ocsvm.fit(X_train)

score_svm = -ocsvm.decision_function(X_test)
pred_svm = np.where(ocsvm.predict(X_test)==-1,1,0)

# LOF
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.08)
```

```
    pred_lof = lof.fit_predict(X_test)
    pred_lof = np.where(pred_lof==-1,1,0)

    # Store unsupervised results
    results_unsupervised.append({
        "ISO_ROC": roc_auc_score(y_test, score_iso),
        "ISO_PR": average_precision_score(y_test, score_iso),
        "ISO_F1": f1_score(y_test, pred_iso),

        "SVM_ROC": roc_auc_score(y_test, score_svm),
        "SVM_PR": average_precision_score(y_test, score_svm),
        "SVM_F1": f1_score(y_test, pred_svm),

        "LOF_F1": f1_score(y_test, pred_lof)
    })


# ============================================================
# 4 AGGREGATE RESULTS
# ============================================================
df_sup = pd.DataFrame(results_supervised)
df_unsup = pd.DataFrame(results_unsupervised)

print("\n===== SUPERVISED RESULTS (Mean) =====")
print(df_sup.mean())

print("\n===== UNSUPERVISED RESULTS (Mean) =====")
print(df_unsup.mean())


# ============================================================
# 5 STATISTICAL SIGNIFICANCE TEST
# ============================================================
t_stat, p_val = ttest_rel(df_sup["RF_F1"], df_sup["LR_F1"])

print("\nPaired t-test RF vs LR (F1)")
print("p-value:", p_val)


# ============================================================
# 6 ROBUSTNESS TEST (NOISE)
# ============================================================
print("\n===== CORRECT ROBUSTNESS TEST =====")

# Train on clean data
rf_clean = RandomForestClassifier()
rf_clean.fit(X, y)

for sigma in [0.05, 0.1, 0.2]:

    noise = np.random.normal(0, sigma, X.shape)
```

```
    X_noisy = X + noise

    pred = rf_clean.predict(X_noisy)

    print(f"Noise {sigma} → F1:", f1_score(y, pred))
```

```
Data Shape: (3000, 15)
Class Distribution: [2750  250]

===== SUPERVISED RESULTS (Mean) =====
LR_ROC    0.830829
LR_PR     0.537074
LR_F1     0.404652
RF_ROC    0.883305
RF_PR     0.657375
RF_F1     0.474059
dtype: float64

===== UNSUPERVISED RESULTS (Mean) =====
ISO_ROC    0.629484
ISO_PR     0.155914
ISO_F1     0.172083
SVM_ROC    0.620604
SVM_PR     0.171502
SVM_F1     0.168371
LOF_F1     0.265306
dtype: float64

Paired t-test RF vs LR (F1)
p-value: 0.1836124071854061

===== CORRECT ROBUSTNESS TEST =====
Noise 0.05 → F1: 0.8235294117647058
Noise 0.1 → F1: 0.7714987714987716
Noise 0.2 → F1: 0.7095115681233933
```

```
import os
print(os.listdir())
```

```
['.config', '.ipynb_checkpoints', 'creditcard.csv',
'creditcard.csv.zip', 'sample_data']
```

```
!pip install shap
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-
packages (0.50.0)
Requirement already satisfied: numpy>=2 in
/usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in
/usr/local/lib/python3.12/dist-packages (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in
```

```
/usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in
/usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in
/usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in
/usr/local/lib/python3.12/dist-packages (from shap) (26.0)
Requirement already satisfied: slicer==0.0.8 in
/usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in
/usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in
/usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in
/usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in
/usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap)
(0.43.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas->shap)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn->shap)
(1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn->shap)
(3.6.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2-
>pandas->shap) (1.17.0)
```

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load dataset
df = pd.read_csv("creditcard.csv")

X = df.drop("Class", axis=1)
y = df["Class"]

# Scale 'Amount'
scaler = StandardScaler()
X["Amount"] = scaler.fit_transform(X[["Amount"]])

# Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)

print("Preprocessing done! Data ready for models.")
```

```
Requirement already satisfied: xgboost in
/usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: numpy in
/usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in
/usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.3)
Requirement already satisfied: scipy in
/usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200:
UserWarning: [17:45:33] WARNING:
/__w/xgboost/xgboost/src/learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)


=== XGBoost Metrics ===
ROC-AUC: 0.9713907060234512
PR-AUC: 0.8361722274299217
```